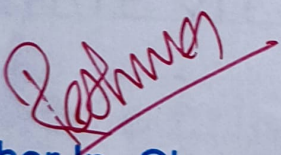


Thadomal Shahani Engineering College

Bandra (W.), Mumbai - 400 050.

❧ CERTIFICATE ❧

Certify that Mr./Miss Aarya Vijay Arban
of IT Department, Semester III with
Roll No. 07 has completed a course of the necessary
experiments in the subject SQL Lab under my
supervision in the **Thadomal Shahani Engineering College**
Laboratory in the year 2023 - 2024

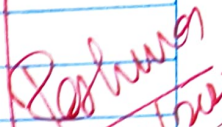
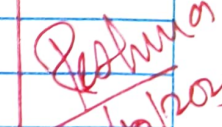
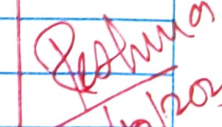

Teacher In- Charge

Head of the Department

Date Oct 30, 2023

Principal

CONTENTS

SR. NO.	EXPERIMENTS	PAGE NO.	DATE	TEACHERS SIGN.
1.	Creating real-world problem statement	1	17/7/23	 30/10/2023
2.	Creating an ER Diagram	2	24/7/23	
3.	Creating an EER Diagram	3	31/7/23	
4.	Creating a schema from EER (Relational Model)	4-5	7/8/23	
5.	Create table/DDL commands	6-13	25/8/23	 30/10/2023
6.	Inserting in table/DML commands	14-26	28/8/23	
7.	Simple SQL queries	27-36	31/8/23	
8.	Complex SQL queries	37-43	4/9/23	
9.	Create view in SQL	44-46	18/9/23	 30/10/2023
10.	Create trigger in SQL	47-50	18/9/23	
11.	Create Function & Procedure	51-53	25/9/23	
12.	Create cursor	54-57	25/9/23	
13.	Written Assignment - 1	58-65	20/10/23	
14.	Written Assignment - 2	66-75	27/10/23	

SQL LAB/SE/S1/07/Assignment No 1

Aim: Write a problem statement for railway reservation system

The problem statement of railway reservation system is to develop and manage a software application that helps us to facilitate the efficient management of ticket bookings, passenger information, transactions details and ticket.

TRAIN: The system should allow users to search for train name, train type, train ticket, train description, train timings, service provided, food provided.

TICKET DETAILS: The ticket details should include name, age and gender of passenger, date and departure timings, travelling distance, quota, start and destination of train travel, seat number.

PASSENGER INFO: The system should store passenger details for each booking like name, age, gender, date of birth, Aadhar number.

SEAT AVAILABILITY: The system should display the number of available seats in different classes (e.g. SL, AC-1, AC-2) for a particular train and also the waiting lists of seats.

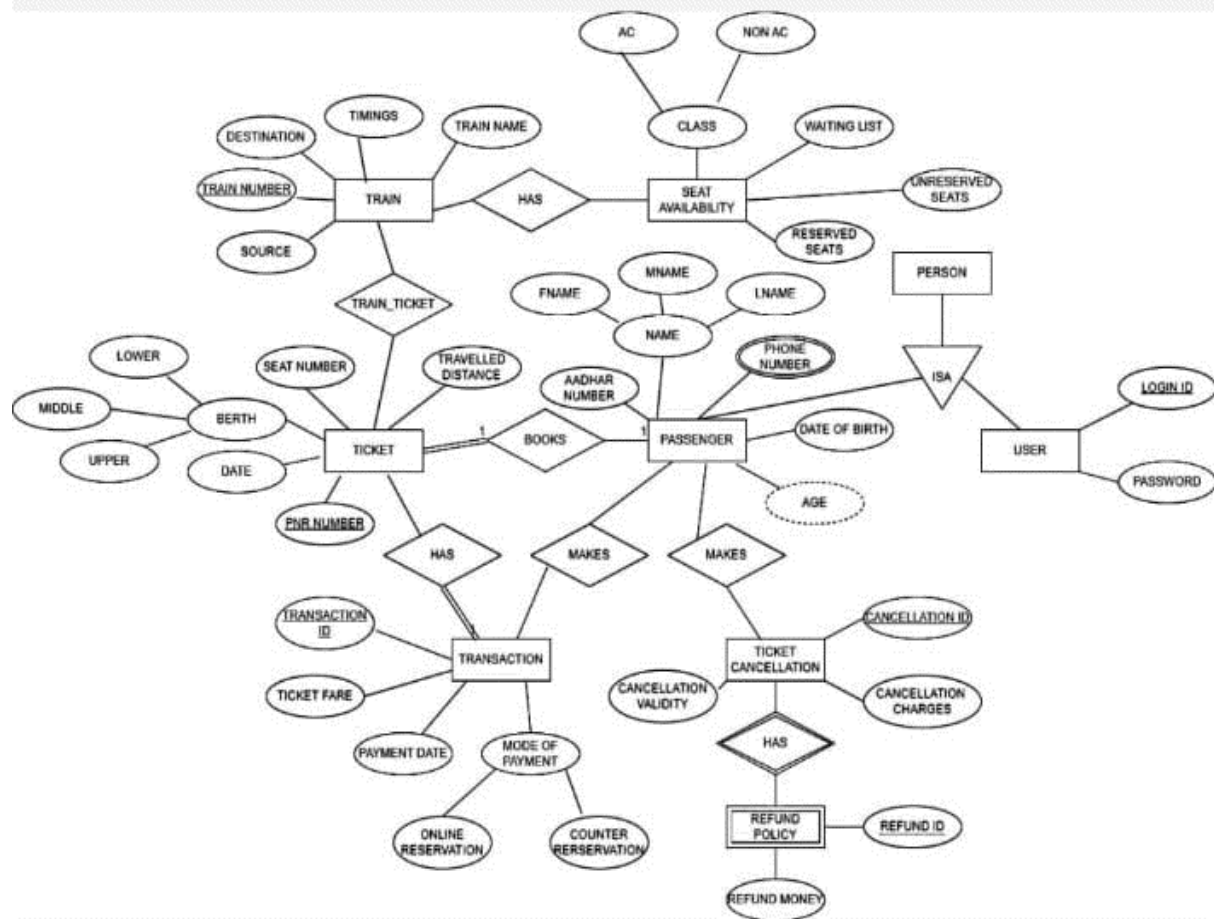
TRANSACTIONS DETAILS: The system should store mode of payment, ticket fare, payment date and time, transaction id.

TICKET CANCELLATION: Passenger should be able to cancel the tickets and cancellation charges should be sanctioned.

Refund: Being dependent on ticket cancellation, it should contain the details to be entered like refund amount, refund validity, mode of refund.

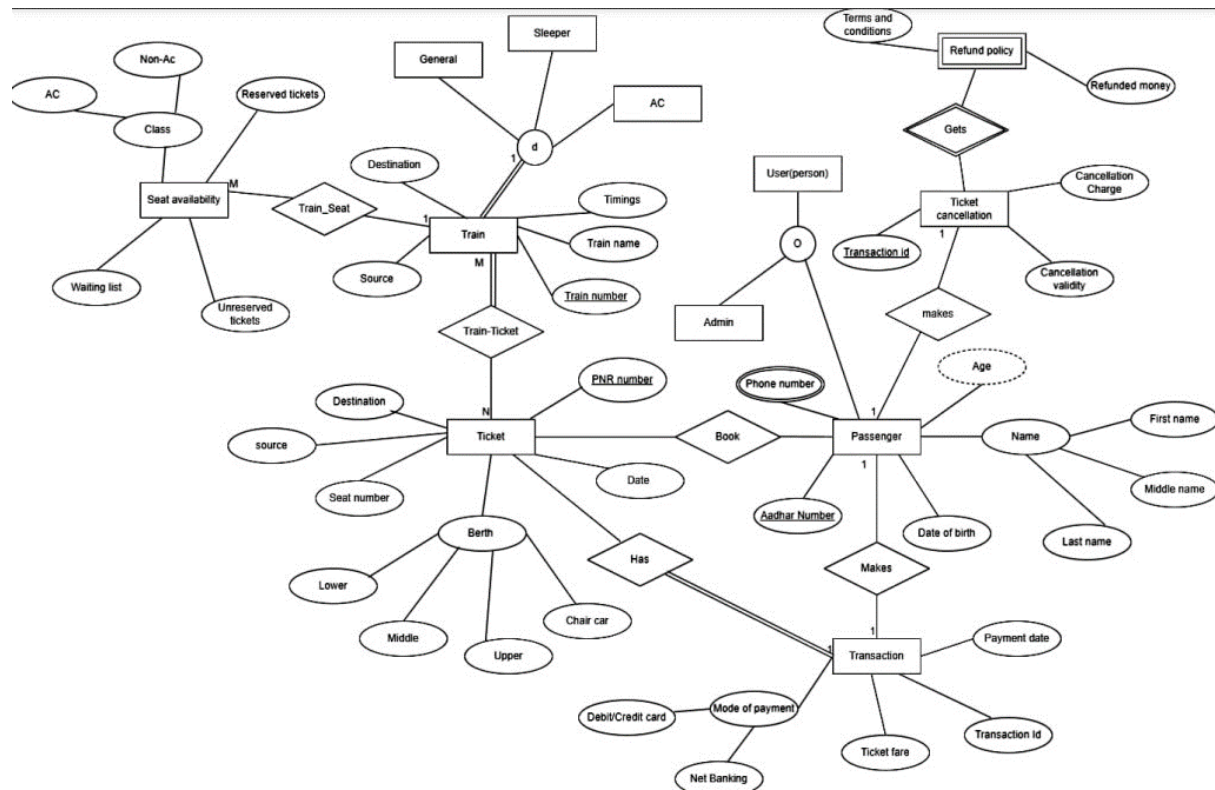
SQL LAB/SE/S1/07/Assignment No 2

Aim: Draw ER Diagram for Railway reservation system



SQL LAB/SE/S1/07/Assignment No 3

Aim: Draw EER Diagram for Railway Reservation System



SQL LAB/SE/S1/07/Assignment No 4

Aim: Convert EER to relation

TRAIN

<u>Train number</u>	Train name	Source	Destination	Timings
---------------------	------------	--------	-------------	---------

GENERAL

<u>TTrain number</u>	Gflag
----------------------	-------

SLEEPER

<u>TTrain number</u>	Sflag
----------------------	-------

AIR CONDITIONER

<u>TTrain number</u>	Aflag
----------------------	-------

SEAT AVAILABILITY

Reserved tickets	Unreserved tickets	Waiting list	AC	Non AC	<u>TTrain number</u>
------------------	--------------------	--------------	----	--------	----------------------

TICKET

<u>PNR number</u>	Source	Destination	Seat number	Date	Lower	Middle	Upper	Chair Car
-------------------	--------	-------------	-------------	------	-------	--------	-------	-----------

SQL LAB/SE/S1/07/Assignment No 4

PERSON

<u>Login id</u>	Password
-----------------	----------

ADMIN

<u>Login id</u>	Admin_Description
-----------------	-------------------

PASSENGER

<u>Aadhar number</u>	Fname	Mname	Lname	Date of birth	mgr_Transaction	<u>PLogin id</u>
----------------------	-------	-------	-------	---------------	-----------------	------------------

PASSENGER_PHONE NUMBER

<u>PAadhar number</u>	Phone number
-----------------------	--------------

PASSENGER_PHONE NUMBER

<u>PAadhar number</u>	Phone Number
-----------------------	--------------

TICKET CANCELLATION

<u>Cancellation id</u>	Cancellation validity	Cancellation charge
------------------------	-----------------------	---------------------

REFUND POLICY

<u>Tcancellation id</u>	<u>Refund id</u>	Refunded	Money	Terms and conditions
-------------------------	------------------	----------	-------	----------------------

TRAIN_TICKET

<u>TTrain Number</u>	<u>TPNR Number</u>
----------------------	--------------------

SQL LAB/SE/S1/07/Assignment No 5

Aim: Create tables with given constraint

Input:

Table 1:

```
mysql> create table client_master_7 (
```

```
-> Client_no varchar(6),  
-> Name varchar(20) NOT NULL,  
-> Address1 varchar(30),  
-> Address2 varchar(30),  
-> City varchar(15),  
-> State varchar(15),  
-> Pincode int(6),  
-> Bal_due float(10,2),  
-> primary key(Client_no),  
-> CHECK (Client_no like 'C%')  
-> );
```

Query OK, 0 rows affected, 2 warnings (0.15 sec)

Table 2:

```
mysql> create table product_master_7 (
```

```
-> Product_no varchar(6),  
-> Description varchar(25) NOT NULL,  
-> Profit_percent float(5,2) NOT NULL,  
-> Unit_measure varchar(10) NOT NULL,  
-> Qty_on_hand int(8),  
-> Reorder_lvl int(8) NOT NULL,  
-> Sell_price float(8,2) NOT NULL,  
-> CHECK (Sell_price<>0),  
-> Cost_price float(8,2) NOT NULL  
-> Check (Cost_price<>0),
```


SQL LAB/SE/S1/07/Assignment No 5

-> primary key(Product_no),

-> CHECK (Product_no like 'P%')

->);

Query OK, 0 rows affected, 5 warnings (0.11 sec)

Table 3:

mysql> create table salesman_master_7

-> (

-> Salesman_no varchar(6),

-> Salesman_name varchar(20) NOT NULL,

-> Address1 varchar(30) NOT NULL,

-> Address2 varchar(30),

-> City varchar(20),

-> Pincode varchar(20),

-> State varchar(20),

-> Sal_amt float(8,2) NOT NULL

-> Check(Sal_amt<>0),

-> Tgt_to_get float(6,2) NOT NULL

-> Check(Tgt_to_get<>0),

-> Ytd_sales float(6,2) NOT NULL,

-> Remarks varchar(60),

-> primary key(Salesman_no),

-> Check (Salesman_no like 'S%')

->);

Query OK, 0 rows affected, 3 warnings (0.75 sec)

Table 4:

create table sales_order_7

-> (

-> S_order_no varchar(6),

SQL LAB/SE/S1/07/Assignment No 5

```
-> S_order_date date,  
-> Client_no varchar(6),  
-> Dely_addr varchar(25),  
-> Salesman_no varchar(6),  
-> Dely_type char(1) CHECK (Dely_type in ('P','F','D')),  
-> Billed_yn char(1),  
-> Dely_date date, CHECK (Dely_date>=S_order_date),  
-> Order_status varchar(10) CHECK (Order_status in ('IP','F','B','C')),  
-> primary key (S_order_no),  
-> foreign key (Client_no) references client_master_7(Client_no),  
-> foreign key (Salesman_no) references salesman_master_7(Salesman_no)  
-> );
```

Query OK, 0 rows affected (1.55 sec)

Table 5:

create table sales_order_details_7

```
-> (  
-> S_order_no varchar(6),  
-> Product_no varchar(6),  
-> Qty_ordered int(8),  
-> Qty_disp int(8),  
-> Product_rate float(10,2),  
-> primary key(S_order_no,Product_no),  
-> foreign key(S_order_no)references sales_order_7(S_order_no),  
-> foreign key(Product_no)references product_master_7(Product_no)  
-> );
```

Query OK, 0 rows affected, 3 warnings (0.27 sec)

Table 6:

create table Challan_Header_7

SQL LAB/SE/S1/07/Assignment No 5

```
-> (  
  -> Challan_no varchar(6),  
  -> S_order_no varchar(6),  
  -> Challan_date date NOT NULL,  
  -> Billed_yn char(1) DEFAULT ('N') check(Billed_yn in ('Y','N')),  
  -> foreign key (S_order_no) references sales_order_7(S_order_no),  
  -> primary key(Challan_no)  
-> );
```

Query OK, 0 rows affected (0.15 sec)

Table 7:

create table Challan_details_7

```
-> (  
  -> Challan_no varchar(6) ,  
  -> Product_no varchar(6) ,  
  -> Qty_disp int(8),  
  -> primary key(Qty_disp) ,  
  -> foreign key(Challan_no) references Challan_Header_7(Challan_no),  
  -> foreign key(Product_no) references product_master_7(Product_no)  
-> );
```

Query OK, 0 rows affected, 1 warning (0.19 sec)

Output:

Table 1:

mysql> desc client_master_7;

Field	Type	Null	Key	Default	Extra
Client_no	varchar(6)	NO	PRI	NULL	
Name	varchar(20)	NO		NULL	

SQL LAB/SE/S1/07/Assignment No 5

```
| Address1 | varchar(30) | YES | | NULL | |
| Address2 | varchar(30) | YES | | NULL | |
| City     | varchar(15) | YES | | NULL | |
| State    | varchar(15) | YES | | NULL | |
| Pincode  | int         | YES | | NULL | |
| Bal_due  | float(10,2) | YES | | NULL | |
```

```
+-----+-----+-----+-----+-----+
```

8 rows in set (0.00 sec)

Table 2:

```
mysql> desc product_master_7;
```

```
+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| Product_no | varchar(6) | NO   | PRI | NULL    |      |
| Description | varchar(25) | NO   |     | NULL    |      |
| Profit_percent | float(5,2) | NO   |     | NULL    |      |
| Unit_measure | varchar(10) | NO   |     | NULL    |      |
| Qty_on_hand | int        | YES  |     | NULL    |      |
| Reorder_lvl | int        | NO   |     | NULL    |      |
| Sell_price  | float(8,2) | NO   |     | NULL    |      |
| Cost_price  | float(8,2) | NO   |     | NULL    |      |
```

```
+-----+-----+-----+-----+-----+
```

8 rows in set (0.01 sec)

Table 3:

```
mysql> desc salesman_master_7;
```

```
+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
```

SQL LAB/SE/S1/07/Assignment No 5

```
| Salesman_no | varchar(6) | NO | PRI | NULL | |
| Salesman_name | varchar(20) | NO | | NULL | |
| Address1 | varchar(30) | NO | | NULL | |
| Address2 | varchar(30) | YES | | NULL | |
| City | varchar(20) | YES | | NULL | |
| Pincode | varchar(20) | YES | | NULL | |
| State | varchar(20) | YES | | NULL | |
| Sal_amt | float(8,2) | NO | | NULL | |
| Tgt_to_get | float(6,2) | NO | | NULL | |
| Ytd_sales | float(6,2) | NO | | NULL | |
| Remarks | varchar(60) | YES | | NULL | |
+-----+-----+-----+-----+-----+
```

11 rows in set (0.16 sec)

Table 4:

```
mysql> desc sales_order_7;
```

```
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| S_order_no | varchar(6) | NO | PRI | NULL | |
| S_order_date | date | YES | | NULL | |
| Client_no | varchar(6) | YES | MUL | NULL | |
| Dely_addr | varchar(25) | YES | | NULL | |
| Salesman_no | varchar(6) | YES | MUL | NULL | |
| Dely_type | char(1) | YES | | NULL | |
| Billed_yn | char(1) | YES | | NULL | |
| Dely_date | date | YES | | NULL | |
| Order_status | varchar(10) | YES | | NULL | |
+-----+-----+-----+-----+-----+
```

SQL LAB/SE/S1/07/Assignment No 5

9 rows in set (0.03 sec)

Table 5:

```
mysql> desc sales_order_details_7;
```

Field	Type	Null	Key	Default	Extra
S_order_no	varchar(6)	NO	PRI	NULL	
Product_no	varchar(6)	NO	PRI	NULL	
Qty_ordered	int	YES		NULL	
Qty_disp	int	YES		NULL	
Product_rate	float(10,2)	YES		NULL	

5 rows in set (0.00 sec)

Table 6:

```
mysql> desc Challan_Header_7;
```

Field	Type	Null	Key	Default	Extra
Challan_no	varchar(6)	NO	PRI	NULL	
S_order_no	varchar(6)	YES	MUL	NULL	
Challan_date	date	NO		NULL	
Billed_yn	char(1)	YES		_cp850\'N\'	DEFAULT_GENERATED

4 rows in set (0.00 sec)

Table 7:

```
mysql> desc Challan_details_7;
```

Field	Type	Null	Key	Default	Extra
-------	------	------	-----	---------	-------

SQL LAB/SE/S1/07/Assignment No 5

+-----+-----+-----+-----+-----+					
Challan_no	varchar(6)	YES	MUL	NULL	
Product_no	varchar(6)	YES	MUL	NULL	
Qty_disp	int	NO	PRI	NULL	
+-----+-----+-----+-----+-----+					

3 rows in set (0.00 sec)

SQL LAB/SE/S1/07/Assignment No 6

Aim: Insert Records to all the tables

Input:

Table 1:

```
mysql> insert into client_master_7 (
```

```
-> Client_no,Name,City,State,Pincode,Bal_due)
```

```
-> values('COOOO1','Ivan Bayross','Bombay','Maharashtra',400054,15000);
```

Query OK, 1 row affected (0.04 sec)

```
mysql> insert into client_master_7 (
```

```
-> Client_no,Name,City,State,Pincode,Bal_due)
```

```
-> values('COOOO2','Vandana Saitwal','Madras','Tamil Nadu',780001,0);
```

Query OK, 1 row affected (0.07 sec)

```
mysql> insert into client_master_7 (
```

```
-> Client_no,Name,City,State,Pincode,Bal_due)
```

```
-> values('COOOO3','Pramada Jaguste','Bombay','Maharashtra',400057,5000);
```

Query OK, 1 row affected (0.04 sec)

```
mysql> insert into client_master_7 (
```

```
-> Client_no,Name,City,State,Pincode,Bal_due)
```

```
-> values('COOOO4','Basu Navindgi','Bombay','Maharashtra',400056,0);
```

Query OK, 1 row affected (0.09 sec)

```
mysql> insert into client_master_7 (
```

```
-> Client_no,Name,City,State,Pincode,Bal_due)
```

```
-> values('COOOO5','Ravi Sreedharan','Delhi',100001,2000);
```

ERROR 1136 (21S01): Column count doesn't match value count at row 1

```
mysql> insert into client_master_7 (
```

SQL LAB/SE/S1/07/Assignment No 6

-> Client_no,Name,City,State,Pincode,Bal_due)

-> values('COOOO5','Ravi Sreedharan','Delhi','',100001,2000);

Query OK, 1 row affected (0.08 sec)

mysql> insert into client_master_7 (

-> Client_no,Name,City,State,Pincode,Bal_due)

-> values('COOOO6','Rukmini','Bombay','Maharashtra',400050,15000);

Query OK, 1 row affected (0.06 sec)

Table 2:

mysql> insert into product_master_7

-> values('POOOO1','1.44 Floppies',5,'Piece',100,20,525,500);

Query OK, 1 row affected (0.03 sec)

mysql> insert into product_master_7

-> values('PO3453','Monitors',6,'Piece',10,3,12000,11280);

Query OK, 1 row affected (0.05 sec)

mysql> insert into product_master_7

-> values('PO6734','Mouse',5,'Piece',20,5,1050,1000);

Query OK, 1 row affected (0.03 sec)

mysql> insert into product_master_7

-> values('PO7865','1.22 Floppies',5,'Piece',100,20,525,500);

Query OK, 1 row affected (0.02 sec)

mysql> insert into product_master_7

-> values('PO7868','Keyboards',2,'Piece',10,3,3150,3050);

Query OK, 1 row affected (0.04 sec)

SQL LAB/SE/S1/07/Assignment No 6

```
mysql> insert into product_master_7
```

```
-> values('PO7885','CD Drive',2.5,'Piece',10,3,5250,5100);
```

Query OK, 1 row affected (0.05 sec)

```
mysql> insert into product_master_7
```

```
-> values('PO7965','HDD',4,'Piece',10,3,8400,8000);
```

Query OK, 1 row affected (0.03 sec)

```
mysql> insert into product_master_7
```

```
-> values('PO7975','1.44 Drive',5,'Piece',10,3,1050,1000);
```

Query OK, 1 row affected (0.07 sec)

```
mysql> insert into product_master_7
```

```
-> values('PO8865','1.22 Drive',5,'Piece',2,3,1050,1000);;
```

Query OK, 1 row affected (0.04 sec)

Table 3:

```
insert into salesman_master_7 values('S00001', 'Kiran', 'A/14', 'Worli', 'Bombay', '400002',  
'MAH', 3000,  
100, 50,'Good');
```

```
insert into salesman_master_7 values('S00002', 'Manish', '65', 'Nariman', 'Bombay',  
'400001',  
'MAH', 3000, 200, 100,'Good');
```

```
insert into salesman_master_7 values('S00003', 'Ravi', 'P-7', 'Bandra', 'Bombay', '400032',  
'MAH',  
3000, 200, 100,'Good');
```

```
insert into salesman_master_7 values('S00004', 'Ashish', 'A/5', 'Juhu', 'Bombay', '400044',  
'MAH', 3000,  
200, 150,'Good');
```

Table 4:

SQL LAB/SE/S1/07/Assignment No 6

```
mysql> insert into sales_order_7
```

```
(S_order_no,S_order_date,Client_no,Salesman_no,Dely_type,Billed_yn,Dely_date,Order_status)
```

```
-> values('O19001','1996-01-12','COOOO1','S00001','F','N','1996-01-20','IP');
```

Query OK, 1 row affected (0.03 sec)

```
mysql> insert into
```

```
sales_order_7(S_order_no,S_order_date,Client_no,Salesman_no,Dely_type,Billed_yn,Dely_date,Order_status)
```

```
-> values('O19002','1996-01-25','COOOO2','S00002','P','N','1996-01-27','C');
```

Query OK, 1 row affected (0.07 sec)

```
mysql> insert into
```

```
sales_order_7(S_order_no,S_order_date,Client_no,Salesman_no,Dely_type,Billed_yn,Dely_date,Order_status)
```

```
-> values('O46865','1996-02-18','COOOO3','S00003','F','Y','1996-02-20','F');
```

Query OK, 1 row affected (0.03 sec)

```
mysql> insert into
```

```
sales_order_7(S_order_no,S_order_date,Client_no,Salesman_no,Dely_type,Billed_yn,Dely_date,Order_status)
```

```
-> values('O19003','1996-04-03','COOOO1','S00001','F','Y','1996-04-07','F');
```

Query OK, 1 row affected (0.04 sec)

```
mysql> insert into
```

```
sales_order_7(S_order_no,S_order_date,Client_no,Salesman_no,Dely_type,Billed_yn,Dely_date,Order_status)
```

```
-> values('O46866','1996-05-20','COOOO4','S00002','P','N','1996-05-22','C');
```

Query OK, 1 row affected (0.01 sec)

SQL LAB/SE/S1/07/Assignment No 6

```
mysql> insert into  
sales_order_7(S_order_no,S_order_date,Client_no,Salesman_no,Dely_type,Billed_yn,Dely_  
date,Order_status)
```

```
-> values('O10008','1996-05-24','COOOO5','S00004','F','N','1996-05-26','IP');
```

Query OK, 1 row affected (0.03 sec)

Table 5:

```
mysql> insert into sales_order_details_7
```

```
-> values ('O19001', 'POOOO1', 4,4,525);
```

Query OK, 1 row affected (0.03 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O19001', 'PO7965', 2,1,8400);
```

Query OK, 1 row affected (0.03 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O19001', 'PO7885', 2,1,5250);
```

Query OK, 1 row affected (0.04 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O19002', 'POOOO1', 10,0,525);
```

Query OK, 1 row affected (0.04 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O46865', 'PO7868', 3,3,3150);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O46865', 'PO7885', 3,1,5250);
```

Query OK, 1 row affected (0.02 sec)

SQL LAB/SE/S1/07/Assignment No 6

```
mysql> insert into sales_order_details_7
```

```
-> values ('O46865', 'POOOO1', 10,10,525);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O46865', 'PO3453', 4,4,1050);
```

Query OK, 1 row affected (0.02 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O19003', 'PO3453', 2,2,1050);
```

Query OK, 1 row affected (0.02 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O19003', 'PO6734', 1,1,12000);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O46866', 'PO7965', 1,0,8400);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O46866', 'PO7975', 1,0,1050);
```

Query OK, 1 row affected (0.02 sec)

```
mysql> insert into sales_order_details_7
```

```
-> values ('O10008', 'POOOO1', 10,5,525);
```

Query OK, 1 row affected (0.11 sec)

SQL LAB/SE/S1/07/Assignment No 6

mysql> insert into sales_order_details_7

-> values ('O10008', 'PO7975', 5,3,1050);

Query OK, 1 row affected (0.03 sec)

Table 6:

mysql> insert into Challan_Header_7

-> values ('CH9001', 'O19001', '1995-12-12', 'Y');

Query OK, 1 row affected (0.06 sec)

mysql> insert into Challan_Header_7

-> values ('CH6865', 'O46865', '1995-11-12', 'Y');

Query OK, 1 row affected (0.01 sec)

mysql> insert into Challan_Header_7

-> values ('CH3965', 'O10008', '1995-11-12', 'Y');

Query OK, 1 row affected (0.04 sec)

Table 7:

mysql> insert into Challan_details_7

-> values('CH9001', 'POOOO1', 4);

Query OK, 1 row affected (0.04 sec)

mysql> insert into Challan_details_7

-> values('CH9001', 'PO7965', 1);

Query OK, 1 row affected (0.04 sec)

mysql> insert into Challan_details_7

-> values('CH6865', 'PO7868', 3);

Query OK, 1 row affected (0.01 sec)

SQL LAB/SE/S1/07/Assignment No 6

```
mysql> insert into Challan_details_7
```

```
-> values('CH6865', 'POOOO1', 10);
```

Query OK, 1 row affected (0.02 sec)

```
mysql> insert into Challan_details_7
```

```
-> values('CH3965', 'POOOO1', 5);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> insert into Challan_details_7
```

```
-> values('CH3965', 'PO7975', 2);
```

Query OK, 1 row affected (0.01 sec)

Output:

Table 1:

```
mysql> select *from client_master_7;
```

Client_no	Name	Address1	Address2	City	State	Pincode	Bal_due
COOOO1	Ivan Bayross	NULL	NULL	Bombay	Maharashtra	400054	15000.00
COOOO2	Vandana Saitwal	NULL	NULL	Madras	Tamil Nadu	780001	0.00
COOOO3	Pramada Jaguste	NULL	NULL	Bombay	Maharashtra	400057	5000.00
COOOO4	Basu Navindgi	NULL	NULL	Bombay	Maharashtra	400056	0.00
COOOO5	Ravi Sreedharan	NULL	NULL	Delhi		100001	2000.00
COOOO6	Rukmini	NULL	NULL	Bombay	Maharashtra	400050	15000.00

6 rows in set (0.00 sec)

SQL LAB/SE/S1/07/Assignment No 6

Table 2:

```
mysql> select *from product_master_7;
```

Product_no	Description	Profit_percent	Unit_measure	Qty_on_hand	Reorder_lvl	Sell_price	Cost_price
PO3453	Monitors	6.00	Piece	10	3	12000.00	11280.00
PO6734	Mouse	5.00	Piece	20	5	1050.00	1000.00
PO7865	1.22 Floppies	5.00	Piece	100	20	525.00	500.00
PO7868	Keyboards	2.00	Piece	10	3	3150.00	3050.00
PO7885	CD Drive	2.50	Piece	10	3	5250.00	5100.00
PO7965	HDD	4.00	Piece	10	3	8400.00	8000.00
PO7975	1.44 Drive	5.00	Piece	10	3	1050.00	1000.00
PO8865	1.22 Drive	5.00	Piece	2	3	1050.00	1000.00
POOOO1	1.44 Floppies	5.00	Piece	100	20	525.00	500.00

9 rows in set (0.00 sec)

Table 3:

```
mysql> select *from salesman_master_7;
```

Salesman_no	Salesman_name	Address1	Address2	City	Pincode	State	Sal_amt	Tgt_to_get	Ytd_sales	Remarks
-------------	---------------	----------	----------	------	---------	-------	---------	------------	-----------	---------

SQL LAB/SE/S1/07/Assignment No 6

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
| S00001 | Kiran | A/14 | Worli | Bombay | 400002 | MAH | 3000.00 |
100.00 | 50.00 | Good |
| S00002 | Manish | 65 | Nariman | Bombay | 400001 | MAH | 3000.00 |
200.00 | 100.00 | Good |
| S00003 | Ravi | P-7 | Bandra | Bombay | 400032 | MAH | 3000.00 |
200.00 | 100.00 | Good |
| S00004 | Ashish | A/5 | Juhu | Bombay | 400044 | MAH | 3000.00 |
200.00 | 150.00 | Good |
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
```

4 rows in set (0.00 sec)

Table 4:

```
mysql> select *from sales_order_7;
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
--+
| S_order_no | S_order_date | Client_no | Dely_addr | Salesman_no | Dely_type |
Billed_yn | Dely_date | Order_status |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
--+
| O10008 | 1996-05-24 | COOOO5 | NULL | S00004 | F | N | 1996-05-26
| IP |
| O19001 | 1996-01-12 | COOOO1 | NULL | S00001 | F | N | 1996-01-20
| IP |
| O19002 | 1996-01-25 | COOOO2 | NULL | S00002 | P | N | 1996-01-27
| C |
| O19003 | 1996-04-03 | COOOO1 | NULL | S00001 | F | Y | 1996-04-07
| F |
| O46865 | 1996-02-18 | COOOO3 | NULL | S00003 | F | Y | 1996-02-20
| F |
| O46866 | 1996-05-20 | COOOO4 | NULL | S00002 | P | N | 1996-05-22
| C |
```

SQL LAB/SE/S1/07/Assignment No 6

```
+-----+-----+-----+-----+-----+-----+-----+-----+
-+
```

6 rows in set (0.00 sec)

Table 5:

```
mysql> select *from sales_order_details_7;
```

```
+-----+-----+-----+-----+-----+
| S_order_no | Product_no | Qty_ordered | Qty_disp | Product_rate |
+-----+-----+-----+-----+-----+
| O10008     | PO7975     | 5           | 3        | 1050.00      |
| O10008     | PO0001     | 10          | 5        | 525.00       |
| O19001     | PO7885     | 2           | 1        | 5250.00      |
| O19001     | PO7965     | 2           | 1        | 8400.00      |
| O19001     | PO0001     | 4           | 4        | 525.00       |
| O19002     | PO0001     | 10          | 0        | 525.00       |
| O19003     | PO3453     | 2           | 2        | 1050.00      |
| O19003     | PO6734     | 1           | 1        | 12000.00     |
| O46865     | PO3453     | 4           | 4        | 1050.00      |
| O46865     | PO7868     | 3           | 3        | 3150.00      |
| O46865     | PO7885     | 3           | 1        | 5250.00      |
| O46865     | PO0001     | 10          | 10       | 525.00       |
| O46866     | PO7965     | 1           | 0        | 8400.00      |
| O46866     | PO7975     | 1           | 0        | 1050.00      |
+-----+-----+-----+-----+-----+
```

14 rows in set (0.00 sec)

Table 6:

```
mysql> select *from Challan_Header_7;
```

```
+-----+-----+-----+-----+
| Challan_no | S_order_no | Challan_date | Billed_yn |
+-----+-----+-----+-----+
```

SQL LAB/SE/S1/07/Assignment No 6

CHALLAN_NO	PRODUCT_NO	CHALLAN_DATE	STATUS
CH3965	O10008	1995-11-12	Y
CH6865	O46865	1995-11-12	Y
CH9001	O19001	1995-12-12	Y

3 rows in set (0.00 sec)

Table 7:

mysql> select *from Challan_details_7;

Challan_no	Product_no	Qty_disp
CH9001	PO7965	1
CH3965	PO7975	2
CH6865	PO7868	3
CH9001	POOOO1	4
CH3965	POOOO1	5
CH6865	POOOO1	10

6 rows in set (0.00 sec)

SQL LAB/SE/S1/07/Assignment No 7

Aim: Perform simple SQL queries

Queries:

Q.1 Find names of all the clients.

```
select Name from client_master_7;
```

```
+-----+
| name  |
+-----+
| Ivan Bayross  |
| Vandana Saitwal |
| Pramada Jaguste |
| Basu Navindgi  |
| Ravi Sreedharan |
| Rukmini       |
```

Q.2 Find names and city of all the clients.

```
select name, city from client_master_7;
```

```
+-----+-----+
| name      | city      |
+-----+-----+
| Ivan Bayross  | Maharashtra |
| Vandana Saitwal | Madras      |
| Pramada Jaguste | Bombay      |
| Basu Navindgi  | Bombay      |
| Ravi Sreedharan | Delhi       |
| Rukmini       | Bombay      |
+-----+-----+
```

Q.3 Print the entire client master table.

```
select *from client_master_7;
```

```
+-----+-----+-----+-----+-----+-----+-----+
```

SQL LAB/SE/S1/07/Assignment No 7

Client_no	Name	Address1	Address2	City	State	Pincode	Bal_due
COOOO1	Ivan Bayross	NULL	NULL	Bombay	Maharashtra	400054	15000.00
COOOO2	Vandana Saitwal	NULL	NULL	Madras	Tamil Nadu	780001	0.00
COOOO3	Pramada Jaguste	NULL	NULL	Bombay	Maharashtra	400057	5000.00
COOOO4	Basu Navindgi	NULL	NULL	Bombay	Maharashtra	400056	0.00
COOOO5	Ravi Sreedharan	NULL	NULL	Delhi		100001	2000.00
COOOO6	Rukmini	NULL	NULL	Bombay	Maharashtra	400050	15000.00

Q.4 Find the names of all the clients having 'A' as 2nd letter in their names.

```
select name from client_master_7
```

```
-> where name like '_a%';
```

name
Vandana Saitwal
Basu Navindgi
Ravi Sreedharan

Q.5 Find out the clients who stay in the city whose 2nd letter is 'a'.

```
select name from client_master_7
```

```
-> where city like '_a%';
```

name
Ivan Bayross

SQL LAB/SE/S1/07/Assignment No 7

| Vandana Saitwal |

+-----+

Q.6 Find the list of all the clients who stay in city 'Bombay' or 'Delhi' or 'Madras'.

select name from client_master_7

-> where (city='bombay' or city= 'madras' or city= 'delhi');

+-----+

| name |

+-----+

| Vandana Saitwal |

| Pramada Jaguste |

| Basu Navindgi |

| Ravi Sreedharan |

| Rukmini |

Q.7 List all the clients who are located in Bombay.

select name from client_master_7

-> where city='bombay';

+-----+

| name |

+-----+

| Pramada Jaguste |

| Basu Navindgi |

| Rukmini |

+-----+

Q.8 Print the list of all clients whose balance due are greater than 10000.

select name , bal_due from client_master_7

-> where bal_due>10000;

+-----+-----+

| name | bal_due |

SQL LAB/SE/S1/07/Assignment No 7

```
+-----+-----+
| Ivan Bayross | 15000.00 |
| Rukmini     | 15000.00 |
+-----+-----+
```

Q.9 Print the information for client: C00001 and C00002

```
select* from client_master_7
```

```
-> where (client_no= 'C00001' or client_no = 'C00002');
```

```
+-----+-----+-----+-----+-----+-----+
| client_no | name      | add1 | add2 | city   | pincode | bal_due |
+-----+-----+-----+-----+-----+-----+
| C00001    | Ivan Bayross | NULL | NULL | Maharashtra | 400054 | 15000.00 |
| C00002    | Vandana Saitwal | NULL | NULL | Madras    | 780001 | 5000.00 |
+-----+-----+-----+-----+-----+-----+
```

Q.10 List the name, city, state of client not in Maharashtra.

```
select name,city,state from client_master_7 where not state=('Maharashtra');
```

```
+-----+-----+-----
----+ | name      | city |
state |
+-----+-----+-----+
| Vandana Saitwal | Madras | Tamil Nadu |
| RaviSreedharan | Delhi  |           |
+-----+-----+-----+
```

Q.11 Find the product with description as 1.44 Drive and 1.22 Drive

```
select Product_no,Description from product_master_7
```

```
-> where (Description='1.22 Drive' or Description='1.44 Drive');
```

```
+-----+-----+
| Product_no | Description |
+-----+-----+
```

SQL LAB/SE/S1/07/Assignment No 7

| PO7975 | 1.44 Drive |

| PO8865 | 1.22 Drive |

+-----+-----+

Q.12 Find the product whose selling price is greater than 2000 and less than or equal to 5000.

select Product_no,Description,Sell_price from product_master_7

-> where (Sell_price between 2000 and 5000);

+-----+-----+-----+

| Product_no | Description | Sell_price |

+-----+-----+-----+

| PO7868 | Keyboards | 3150.00 |

+-----+-----+-----+

Q.13 Find the product whose selling price is greater than 1500 and also find its new price as Selling price*15.

select Product_no,Description,Sell_price,Sell_price*15 as New_Price from
product_master_7

-> where (Sell_price>1500);

+-----+-----+-----+-----+

| Product_no | Description | Sell_price | New_Price |

+-----+-----+-----+-----+

| PO3453 | Monitors | 12000.00 | 180000.00 |

| PO7868 | Keyboards | 3150.00 | 47250.00 |

| PO7885 | CD Drive | 5250.00 | 78750.00 |

| PO7965 | HDD | 8400.00 | 126000.00 |

+-----+-----+-----+-----+

Q.14 Find the product whose cost price is less than 1500.

select product_no,description,cost_price from product_master_7

-> where Cost_price<1500;

+-----+-----+-----+

SQL LAB/SE/S1/07/Assignment No 7

product_no	description	cost_price
PO6734	Mouse	1000.00
PO7865	1.22 Floppies	500.00
PO7975	1.44 Drive	1000.00
PO8865	1.22 Drive	1000.00
POOOO1	1.44 Floppies	500.00

Q.15 List the product in sorted order of their description.

select description from product_master_7

-> Order by description;

description
1.22 Drive
1.22 Floppies
1.44 Drive
1.44 Floppies
CD Drive
HDD
Keyboards
Monitors
Mouse

Q.16 Divide the cost price of product HDD by difference between its price and 100 and rename it as Difference.

select cost_price/(cost_price-100) as Difference from product_master_7

-> where description = 'HDD';

SQL LAB/SE/S1/07/Assignment No 7

```
+-----+
| Difference |
+-----+
| 1.012658 |
+-----+
```

Q.17 List the product number, description, sell price of a product whose description begins with letter 'M'.

```
select product_no,description,sell_price from product_master_7
```

```
-> where description like 'M%';
```

```
+-----+-----+-----+
| product_no | description | sell_price |
+-----+-----+-----+
| PO3453     | Monitors   | 12000.00   |
| PO6734     | Mouse      | 1050.00    |
+-----+-----+-----+
```

Q.18 List all the orders that were cancelled in the month of January.

```
select * from sales_order_7
```

```
-> where Order_status='C'and S_order_date between '1996-01-01' and '1996-01-31';
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-+
| S_order_no | S_order_date | Client_no | Dely_addr | Salesman_no | Dely_type | Billed_yn | Dely_date | Order_status |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-+
| O19002     | 1996-01-25   | COOOO2   | NULL      | S00002      | P         | N         | 1996-01-27 | C             |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-+
```

Q.19 List all the products available from product master.

```
Select Description from product_master_7;
```

SQL LAB/SE/S1/07/Assignment No 7

```
+-----+
| description |
+-----+ |

1.44
Floppies |
|
Monitors
|

| Mouse    |
| 1.22 Floppies |
| Keyboards  |
| CD Drive   |
| HDD        |
| 1.44 Drive  |
| 1.22 Drive  |
+-----+
```

Q.20 Calculate square root of cost price and arrange it in increasing order.

```
select product_no,description,cost_price,sqrt(cost_price) as SquareRoot from
product_master order by cost_price;
```

```
+-----+-----+-----+-----+
| product_no | description | cost_price | SquareRoot |
+-----+-----+-----+-----+
| P00001    | 1.44 Floppies | 500.00    | 22.360679774997898 |
| P07865    | 1.22 Floppies | 500.00    | 22.360679774997898 |
| P06734    | Mouse        | 1000.00   | 31.622776601683793 |
| P07975    | 1.44 Drive   | 1000.00   | 31.622776601683793 |
```

SQL LAB/SE/S1/07/Assignment No 7

P08865	1.22 Drive	1000.00	31.622776601683793	
P07868	Keyboards	3050.00	55.226805085936306	
P07885	CD Drive	5100.00	71.4142842854285	
P07965	HDD	8000.00	89.44271909999159	
P03453	Monitors	11280.00	106.20734437881403	

+-----+-----+

Q.21 Print the information from the sales order table of order placed in the month of January.

```
select * from sales_order where s_order_date like '%-01-%';
```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-+

S_order_no	S_order_date	Client_no	Dely_addr	Salesman_no	Dely_type	
Billed_yn	Dely_date	Order_status				

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-+

O19001	1996-01-12	C00001	NULL	S00001	F	N	1996-01-20	
IP								

O19002	1996-01-25	C00002	NULL	S00002	P	N	1996-01-27	
C								

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-+

Q.22 List all the orders that were cancelled in the month of March.

```
select * from sales_order_7
```

```
-> where Order_status='C'and S_order_date between '1996-03-01' and '1996-03-31';
```

Empty set (0.01 sec)

SQL LAB/SE/S1/07/Assignment No 8

Aim: Implementation of Complex Queries

Queries:

Q.1 Count the total number of orders.

```
select count(*) from sales_order_7;
```

```
+-----+
| count(*) |
+-----+
|      6 |
+-----+
```

Q.2 Calculate average price of all products.

```
select round(avg(sell_price)) from product_master_7;
```

```
+-----+
| round(avg(sell_price)) |
+-----+
|           3667 |
+-----+
```

Q.3 Calculate minimum price of product.

```
select min(sell_price) from product_master_7;
```

```
+-----+
| min(sell_price) |
+-----+
|       525.00 |
+-----+
```

Q.4 Determine maximum and minimum product price and rename its title Max_price and Min_price respectively.

```
select min(cost_price) as Min_price, max(cost_price) as Max_price from product_master_7;
```

SQL LAB/SE/S1/07/Assignment No 8

```
+-----+-----+
| Min_price | Max_price |
+-----+-----+
|  500.00 | 11280.00 |
+-----+-----+
```

Q.5 Count the number of products having price greater than or equal to 1500.

```
select count(*) from product_master_7
-> where sell_Price >= 1500;
```

```
+-----+
| count(*) |
+-----+
|      4 |
+-----+
```

Q.6 Find all the products whose quantity oh hand is less than recorder level.

```
select description from product_master_7
-> where Qty_on_hand < Reorder_lvl;
```

```
+-----+
| description |
+-----+
| 1.22 Drive |
+-----+
```

Q.7 Print the description and total quantity sold for each product.

```
select s.Product_no, p.Description, sum(s.Qty_ordered)
-> From sales_order_details_7 s, product_master_7 p
-> where p.Product_no = s.Product_no
-> Group by s.Product_no , p.Description;
```

```
+-----+-----+-----+
| Product_no | Description | sum(s.Qty_ordered) |
```


SQL LAB/SE/S1/07/Assignment No 8

+-----+-----+-----+		
PO7975	1.44 Drive	6
PO0001	1.44 Floppies	34
PO7885	CD Drive	5
PO7965	HDD	3
PO3453	Monitors	6
PO6734	Mouse	1
PO7868	Keyboards	3
+-----+-----+-----+		

Q.8 Find the value of each product sold.

```
select s.Product_no,p.Description,sum(s.Qty_disp*s.Product_rate)
```

-> From sales_order_details_7 s, product_master_7 p

-> where p.Product_no = s.Product_no

-> Group by s.Product_no , p.Description;

+-----+-----+-----+		
Product_no	Description	sum(s.Qty_disp*s.Product_rate)
+-----+-----+-----+		
PO7975	1.44 Drive	3150.00
PO0001	1.44 Floppies	9975.00
PO7885	CD Drive	10500.00
PO7965	HDD	8400.00
PO3453	Monitors	6300.00
PO6734	Mouse	12000.00
PO7868	Keyboards	9450.00
+-----+-----+-----+		

Q.9 Find the product number and description of non-moving product.

```
select product_no,description from product_master_7
```

-> where product_no not in (select product_no from sales_order_details_7);

SQL LAB/SE/S1/07/Assignment No 8

```
+-----+-----+
| product_no | description |
+-----+-----+
| PO7865     | 1.22 Floppies |
| PO8865     | 1.22 Drive   |
+-----+-----+
```

Q.10 Find the customer name, city and pin-code for the client who has placed the order number (019001).

```
select Name,City,Pincode from client_master_7
```

```
-> where Client_no IN (select client_no from sales_order_7 where s_order_no =
'O19001');
```

```
+-----+-----+-----+
| Name      | City   | Pincode |
+-----+-----+-----+
| Ivan Bayross | Bombay | 400054 |
+-----+-----+-----+
```

Q.11 Find the product which has been sold to Ivan Bayross.

```
select d. product_no,p.description From sales_order_details_7 d,product_master_7 p,
client_master_7 c, sales_order_7 s
```

```
-> where p.product_no = d.product_no and s.S_order_no = d.S_order_no and c.ClientT_no
= s.ClientT_no and c.Name ='Ivan Bayross';
```

```
+-----+-----+
| product_no | description |
+-----+-----+
| PO7885     | CD Drive   |
| PO7965     | HDD        |
| PO0001     | 1.44 Floppies |
| PO3453     | Monitors   |
| PO6734     | Mouse      |
```

SQL LAB/SE/S1/07/Assignment No 8

+-----+-----+

Q.12 Find the names of the clients who have purchased 'CD Drive'.

```
select distinct s.Client_no,c.Name from sales_order_details_7 d, sales_order_7 s,
product_master_7 p, client_master_7 c where p.Product_no=d.Product_no and
s.S_order_no=d.S_order_no and c.Client_no=s.Client_no
and p.Description='CD Drive';
```

+-----+-----+

Client_no	Name
-----------	------

+-----+-----+

C00001	Ivan Bayross
--------	--------------

C00003	Vandana Saitwal
--------	-----------------

Q.13 List the product_no and s_order_no of customers having qty_ordered less than 5 from the

order detail Table for the product '1.44 Floppies'.

```
select d.Product_no,d.S_order_no from sales_order_details_7 d,sales_order_7
s,product_master_7 p
where s.S_order_no=d.S_order_no and p.Product_no=d.Product_no and
d.Qty_ordered<5 and p.Description='1.44 Floppies';
```

+-----+-----+

Product_no	S_order_no
------------	------------

+-----+-----+

P00001	O19001
--------	--------

+-----+-----+

Q.14 Find the products and their quantities for the orders placed by client number 'C00001' and 'C00002'.

```
select s.Client_no,d.Product_no,p.Description ,sum(Qty_ordered)"Qty_ordered"
from
sales_order_7 s, sales_order_details_7 d, product_master_7 p, client_master_7 c where
s.s_order_no=d.s_order_no and d.Product_no=p.Product_no and
s.Client_no=c.Client_no
```

SQL LAB/SE/S1/07/Assignment No 8

group by s.Client_no,d.Product_no,p.Description having
s.Client_no='C00001' or s.Client_no='C00002';

Client_no	Product_no	Description	Qty_ordered
C00001	P00001	1.44 Floppies	4
C00001	P07885	CD Drive	2
C00001	P07965	HDD	2
C00001	P03453	Monitors	2
C00001	P06734	Mouse	1
C00002	P00001	1.44 Floppies	10

Q.15 Find the product number and description of moving products.

select distinct p.Product_no,p.Description from product_master_7 p ,sales_order_details_7
d where p.Product_no=d.Product_no;

Product_no	Description
P00001	1.44 Floppies
P03453	Monitors
P06734	Mouse
P07868	Keyboards
P07885	CD Drive
P07965	HDD
P07975	1.44 Drive

SQL LAB/SE/S1/07/Assignment No 9

Aim: Create view using single table and multiple tables.

Single Table:

create view v1 as select client_no,name,city from client_master_7;

Query OK, 0 rows affected (0.50 sec)

select * from v1;

client_no	name	city
COOOO1	Ivan Bayross	Bombay
COOOO2	Vandana Saitwal	Madras
COOOO3	Pramada Jaguste	Bombay
COOOO4	Basu Navindgi	Bombay
COOOO5	Ravi Sreedharan	Delhi
COOOO6	Rukmini	Bombay

Multiple Table:

create view v2 as select p.product_no,c.client_no from client_master_7 c, product_master_7 p;

Query OK, 0 rows affected (0.03 sec)

select * from v2;

product_no	client_no
PO3453	COOOO6
PO3453	COOOO5
PO3453	COOOO4
PO3453	COOOO3
PO3453	COOOO2

SQL LAB/SE/S1/07/Assignment No 9

PO3453	COOOO1	
PO6734	COOOO6	
PO6734	COOOO5	
PO6734	COOOO4	
PO6734	COOOO3	
PO6734	COOOO2	
PO6734	COOOO1	
PO7865	COOOO6	
PO7865	COOOO5	
PO7865	COOOO4	
PO7865	COOOO3	
PO7865	COOOO2	
PO7865	COOOO1	
PO7868	COOOO6	
PO7868	COOOO5	
PO7868	COOOO4	
PO7868	COOOO3	
PO7868	COOOO2	
PO7868	COOOO1	
PO7885	COOOO6	
PO7885	COOOO5	
PO7885	COOOO4	
PO7885	COOOO3	
PO7885	COOOO2	
PO7885	COOOO1	
PO7965	COOOO6	
PO7965	COOOO5	
PO7965	COOOO4	

SQL LAB/SE/S1/07/Assignment No 9

PO7965	CO0003	
PO7965	CO0002	
PO7965	CO0001	
PO7975	CO0006	
PO7975	CO0005	
PO7975	CO0004	
PO7975	CO0003	
PO7975	CO0002	
PO7975	CO0001	
PO8865	CO0006	
PO8865	CO0005	
PO8865	CO0004	
PO8865	CO0003	
PO8865	CO0002	
PO8865	CO0001	
PO0001	CO0006	
PO0001	CO0005	
PO0001	CO0004	
PO0001	CO0003	
PO0001	CO0002	
PO0001	CO0001	

+-----+-----+

54 rows in set (0.04 sec)

SQL LAB/SE/S1/07/Assignment No 10

Aim: Create Trigger

CREATE TABLE Student

```
-> (  
-> roll_no int(2) NOT NULL,  
-> name varchar(20) not null,  
-> city varchar(15) not null,  
-> state varchar(20) not null  
-> );
```

Query OK, 0 rows affected, 1 warning (0.27 sec)

insert into student (roll_no,name,city,state)

```
-> values(07,'Aarya Arban','Mumbai','Maharashtra');
```

Query OK, 1 row affected (0.06 sec)

Trigger 1:

delimiter \$\$

mysql> create trigger tg1

```
-> before insert on Student  
-> for each row  
-> begin  
-> set new.name = upper(new.name);  
-> set new.city = upper(new.city);  
-> end;  
-> $$
```

Query OK, 0 rows affected (0.15 sec)

insert into Student (roll_no,name,city,state)

```
-> values(02,'Shania','Mumbai','Maharashtra');  
-> $$
```

Query OK, 1 row affected (0.07 sec)

select * from Student;

SQL LAB/SE/S1/07/Assignment No 10

-> \$\$

```
+-----+-----+-----+-----+
| roll_no | name      | city  | state  |
+-----+-----+-----+-----+
| 7 | Aarya Arban | Mumbai | Maharashtra | ---- Before Trigger
| 2 | SHANIA     | MUMBAI | Maharashtra | ----- After Trigger
```

```
+-----+-----+-----+-----+
```

create table person (

-> fname char(10),

-> lname char(10),

-> id decimal(8) primary key

->);

Query OK, 0 rows affected (1.08 sec)

mysql> insert into person values ('leena','kirtikar',43);

Query OK, 1 row affected (0.10 sec)

mysql> insert into person values ('aakash','kapadia',38);

Query OK, 1 row affected (0.06 sec)

mysql> insert into person values ('parth','shroff',63);

Query OK, 1 row affected (0.02 sec)

select * from person;

```
+-----+-----+-----+
| fname | lname  | id |
+-----+-----+-----+
| aakash | kapadia | 38 |
| leena  | kirtikar | 43 |
| parth  | shroff  | 63 |
```

SQL LAB/SE/S1/07/Assignment No 10

+-----+-----+-----+

```
create table audit_log(
```

```
  ofname char(10),
```

```
  olname char(10),
```

```
  nfname char(10),
```

```
  nname char(10)
```

```
,cwhen date);
```

Query OK, 0 rows affected (0.16 sec)

Trigger 2:

```
delimiter $$
```

```
mysql> create trigger t2
```

```
  -> after update on person
```

```
  -> for each row
```

```
  -> begin
```

```
  -> insert into audit_log
```

```
  -> values(old.fname,old.lname,new.fname,new.lname,curdate())
```

```
  -> );
```

```
  -> end;
```

```
  -> $$
```

Query OK, 0 rows affected (0.13 sec)

```
mysql> update person set fname='aman' where id like 38;
```

```
  -> $$
```

Query OK, 1 row affected (0.05 sec)

Rows matched: 1 Changed: 1 Warnings: 0

```
mysql> update person set fname='anvay' where id like 63;
```

```
  -> $$
```

Query OK, 1 row affected (0.04 sec)

SQL LAB/SE/S1/07/Assignment No 10

Rows matched: 1 Changed: 1 Warnings: 0

```
mysql> update person set fname='anish' where id like 43;
```

```
-> $$
```

Query OK, 0 rows affected (0.03 sec)

Rows matched: 1 Changed: 0 Warnings: 0

```
mysql> select * from person;
```

```
-> $$
```

fname	lname	id
aman	kapadia	38
anish	kirtikar	43
anvay	shroff	63

3 rows in set (0.00 sec)

SQL LAB/SE/S1/07/Assignment No 11

Aim: Write PL/SQL code to create function and procedure

Queries:

```
mysql> create procedure my_pro11(OUT client_no int)
```

```
-> begin
```

```
-> select COUNT(*) INTO client_no FROM client_master_7;
```

```
-> end;
```

```
-> &&
```

Query OK, 0 rows affected (0.07 sec)

```
mysql> call my_pro(2);
```

```
-> &&
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| client_no | name      | address1 | address2 | city  | state  | pincode | bal_due |
+-----+-----+-----+-----+-----+-----+-----+-----+
| C00001    | Vandana Saitwal | NULL     | NULL     | Madras | Tamil Nadu | 780001 | 0.00 |
| C00002    | Vandana Saitwal | NULL     | NULL     | Madras | Tamil Nadu | 780001 | 0.00 |
```

```
mysql> call my_pro11(@a);
```

```
-> &&
```

Query OK, 1 row affected (0.03 sec)

```
mysql> select @a;
```

```
-> &&
```

```
+-----+
```

```
| @a  |
```

```
+-----+
```

```
| 6  |
```

```
+-----+
```

```
mysql> delimiter $$
```

```
mysql> create function proprice(sell_price double)RETURNS varchar(20)
```

SQL LAB/SE/S1/07/Assignment No 11

-> DETERMINISTIC

-> BEGIN

-> DECLARE lvl varchar(20);

-> IF sell_price<1000 THEN

-> SET lvl='CHEAP';

-> ELSEIF sell_price>3000 THEN

-> SET lvl='EXPENSIVE';

-> END IF;

-> RETURN(lvl);

-> END;

-> \$\$

Query OK, 0 rows affected (0.06 sec)

mysql> select product_no,proprice(sell_price) from product_master_7;

-> \$\$

product_no	proprice(sell_price)
PO3453	EXPENSIVE
PO6734	NULL
PO7865	CHEAP
PO7868	EXPENSIVE
PO7885	EXPENSIVE
PO7965	EXPENSIVE
PO7975	NULL
PO8865	NULL
POOOO1	CHEAP

9 rows in set (0.02 sec)

SQL LAB/SE/S1/07/Assignment No 11

```
mysql> select product_no,proprice(sell_price) from product_master_7;
```

```
-> $$
```

```
+-----+-----+
| product_no | proprice(sell_price) |
+-----+-----+
| PO3453     | EXPENSIVE            |
| PO6734     | NULL                 |
| PO7865     | CHEAP                |
| PO7868     | EXPENSIVE            |
| PO7885     | EXPENSIVE            |
| PO7965     | EXPENSIVE            |
| PO7975     | NULL                 |
| PO8865     | NULL                 |
| POOOO1     | CHEAP                |
+-----+-----+
```

```
9 rows in set (0.02 sec)
```

SQL LAB/SE/S1/07/Assignment No 12

Aim: Write PL/SQL code to create Cursor

Queries:

```
mysql> delimiter $$
```

```
mysql> CREATE procedure CPr()
```

```
-> BEGIN
```

```
-> DECLARE done INT(10) DEFAULT 0;
```

```
-> DECLARE i varchar(20);
```

```
-> DECLARE n varchar(100);
```

```
-> declare curs1 CURSOR FOR select roll_no,name from Student;
```

```
-> declare CONTINUE HANDLER FOR NOT FOUND set done=1;
```

```
-> OPEN curs1;
```

```
-> read_loop:LOOP
```

```
-> fetch curs1 INTO i,n;
```

```
-> If done=1 THEN
```

```
-> leave read_loop;
```

```
-> end if;
```

```
-> select i,n;
```

```
-> end loop read_loop;
```

```
-> close curs1;
```

```
-> end;
```

```
-> $$
```

Query OK, 0 rows affected, 1 warning (0.07 sec)

```
mysql> CALL CPr();
```

```
-> $$
```

```
+-----+-----+
```

```
| i | n |
```

```
+-----+-----+
```

```
| 7 | Aarya Arban |
```

SQL LAB/SE/S1/07/Assignment No 12

```
+-----+-----+
```

1 row in set (0.03 sec)

```
+-----+-----+
```

```
| i | n |
```

```
+-----+-----+
```

```
| 2 | SHANIA |
```

```
+-----+-----+
```

1 row in set (0.03 sec)

Query OK, 0 rows affected (0.04 sec)

```
mysql> update person set email='aman@gmail.com' where id like 38;
```

```
update person set email='anish@gmail.com' where id like 43;
```

```
update person set email='anvay@gmail.com' where id like 63;
```

```
-> $$
```

Query OK, 1 row affected (0.04 sec)

Rows matched: 1 Changed: 1 Warnings: 0

Query OK, 1 row affected (0.09 sec)

Rows matched: 1 Changed: 1 Warnings: 0

Query OK, 1 row affected (0.10 sec)

Rows matched: 1 Changed: 1 Warnings: 0

```
mysql> select * from person;
```

```
-> $$
```

```
+-----+-----+-----+-----+
```

```
| fname | lname | id | email |
```

```
+-----+-----+-----+-----+
```

```
| aman | kapadia | 38 | aman@gmail.com |
```

```
| anish | kirtikar | 43 | anish@gmail.com |
```

```
| anvay | shroff | 63 | anvay@gmail.com |
```


SQL LAB/SE/S1/07/Assignment No 12

+-----+-----+-----+-----+

3 rows in set (0.00 sec)

mysql> delimiter \$\$

mysql> CREATE procedure build_email_list(INOUT email_list varchar(400))

-> BEGIN

-> DECLARE v_finished INTEGER DEFAULT 0;

-> declare v_email varchar(100) DEFAULT "";

-> DECLARE email_cursor CURSOR FOR

-> SELECT email from person;

-> DECLARE CONTINUE HANDLER

-> FOR NOT FOUND set v_finished=1;

-> OPEN email_cursor;

-> get_email:LOOP

-> FETCH email_cursor INTO v_email;

-> IF v_finished=1 THEN

-> LEAVE get_email;

-> END IF;

-> SET email_list=CONCAT(v_email,";",email_list);

-> END LOOP get_email;

-> CLOSE email_cursor;

-> END;

-> \$\$

Query OK, 0 rows affected (0.05 sec)

mysql> SET @email_list="";

-> \$\$

Query OK, 0 rows affected (0.02 sec)

mysql> CALL build_email_list(@email_list);

-> \$\$

SQL LAB/SE/S1/07/Assignment No 12

Query OK, 0 rows affected (0.00 sec)

```
mysql> select @email_list;
```

-> \$\$

```
+-----+
| @email_list          |
+-----+
| anvay@gmail.com;anish@gmail.com;aman@gmail.com; |
+-----+
```

1 row in set (0.00 sec)

```
mysql> SET @email_list="2";$$
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> CALL build_email_list(@email_list);$$
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> select @email_list;$$
```

```
+-----+
| @email_list          |
+-----+
| anvay@gmail.com;anish@gmail.com;aman@gmail.com;2 |
+-----+
```

1 row in set (0.00 sec)

SERIALIZABILITY

A schedule is serializable if it is equal to equivalent to a serial schedule. Different forms of schedule equivalence give rise to notion of:

- 1) Conflict serializability
- 2) View serializability

Conflicting Instructions:

- 1) $I_i = \text{read}$ $I_j = \text{read}$ - Non conflicting
- $I_i = \text{read}$ $I_j = \text{write}$ } Conflicting
- $I_i = \text{write}$ $I_j = \text{read}$ }
- $I_i = \text{write}$ $I_j = \text{write}$ }

We say that schedule S is called a conflict serializable if it is conflict equivalent to serial schedule.

Problem 1:

Check whether the schedule is serializable or not

T_1
 $R(A)$

T_2

$R(A)$

T_3

$R(A)$

T_4

$R(A)$

$W(B)$

$W(B)$

$W(B)$

$W(B)$

Checking whether s is conflicting serializable or not

→ Step 1:

$W_1(B), W_2(B)$

$(T_1 \rightarrow T_2)$

$W_1(B), W_3(B)$

$(T_1 \rightarrow T_3)$

$W_1(B), W_4(B)$

$(T_1 \rightarrow T_4)$

$W_2(B), W_3(B)$

$(T_2 \rightarrow T_3)$

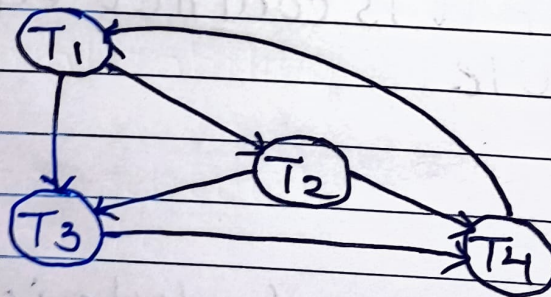
$W_3(B), W_4(B)$

$(T_3 \rightarrow T_4)$

$W_2(B), W_4(B)$

$(T_2 \rightarrow T_4)$

Step 2:



Clearly there exists no cycle in presence so, the given schedule s is connecting schedule.

Hence, it is also a view schedule.

CONCURRENCY PROBLEMS IN DBMS

Concurrency problems are

Concurrency problems in Transaction.

- Dirty Read Problem
- Unrepeatable Read Problem
- Least Update Problem
- Phantom Problem

* Least Update Problem

A write without performing a read operation is called blind write.

serializability scheduler behave exactly as same as serial scheduler

Thus, they are always:

- 1) Consistent
- 2) Reliable
- 3) Cascadess
- 4) Strict

Check whether a schedule is conflict serializable or not:

- 1) Find all list of conflicting operation
- 2) Start creating a precedence graph by drawing one node for each transaction.
- 3) Draw an edge for each conflict pair such that $X_i(V)$ & $Y_j(V)$ forms a conflict pair then draw an edge from T_i to T_j . Thus ensure that T_i gets executed before T_j .
- 4) Check if there is only one cycle formed in the graph. If there is no cycle formed, the graph is conflict serializable else not.

View Serializability

If a given schedule is found to be view equivalent to some serial schedule, then it is called as view serializable schedule.

View Equivalent Schedules

Condition 1:

For each data item X , if transaction T_i reads X from the database initially in schedule S_1 , then in schedule S_2 also, T_i must perform the initial read of X from the database.

Condition 2:

If transaction T_i reads a data item that has been updated by the transaction T_j in schedule S_1 , then in schedule S_2 also, transaction T_i must read the same data item that has been updated by the transaction T_j .

Condition 3:

For each data item X , if X has been updated at last by transaction T_i in schedule S_1 , then in schedule S_2 also, X must be updated at last by transaction T_i .

Checking Whether a Schedule is View Serializable or Not:

Method 1:

- ① Check whether the given schedule is conflict serializable or not.
- ② If the given schedule is conflict serializable, then it is surely view serializable.
- ③ If the given schedule is not conflict serializable, then it may or may not be view serializable. Go and check using other methods.

Method 2:

- ① Check if there exists any blind write operation.
- ② If there does not exist any blind write, then the schedule is surely not view serializable.
- ③ If there exists any blind write, then the schedule may or may not be view serializable.

Method 3:

- ① In this method, try finding a view equivalent serial schedule.
- ② Using above 3 conditions, write all dependencies.
- ③ Draw a graph using dependencies.
- ④ If there exists no cycle in graph, the schedule is view serializable else not.

Problem - 02.

Check whether the given schedule S is view serializable or not.

T_1
 $R(A)$

T_2
 $R(A)$

T_3
 $W(A)$

$W(A)$

→ Step 1: (Method 1)

$R_1(A), W_3(A)$

$R_2(A), W_3(A)$

$W_3(A), W_1(A)$

$R_2(A), W_1(A)$

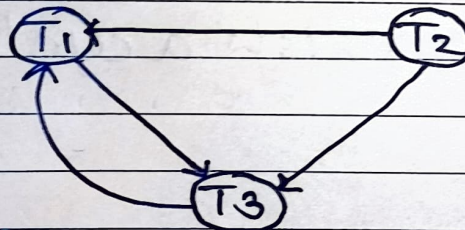
$(T_1 \rightarrow T_3)$

$(T_2 \rightarrow T_3)$

$(T_3 \rightarrow T_1)$

$(T_2 \rightarrow T_1)$

Step 2:



As cycle exists, it is not conflict serializable.

Method 2:

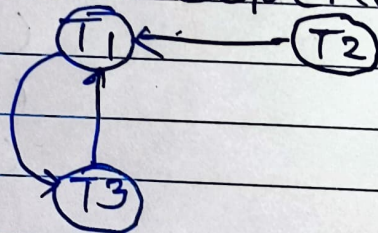
There is a blind write between $W_3(A)$ & $W_1(A)$.

Since, $W_1(A)$ & $W_3(A)$ are conflict pairs, Method 2 is invalid.

Method 3:

Drawing a Dependency Graph:

- ① T1 firstly reads A and T3 firstly updates A. So, T1 must execute before T3
- ② Thus, we get the dependency $T_1 \rightarrow T_3$
- ③ Finally, updating on A is made by the transaction T1. So, T1 must execute after all other transactions
- ④ Thus, we get the dependency $(T_2, T_3) \rightarrow T_1$. There exists no write-read sequences
- ⑤ Drawing the dependency graph:



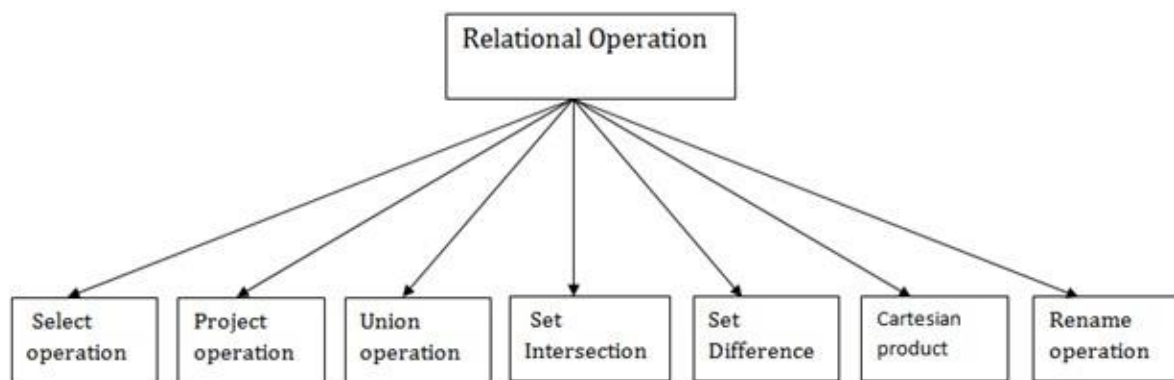
Clearly, there exists a cycle in the dependency

Written Assignment - 2

Relational Algebra

- Relational algebra is a procedural query language.
- It gives a step by step process to obtain the result of the query.
- It uses operators to perform queries.

Types of Relational operation



1. Select Operation

- The select operation selects tuples that satisfy a given predicate.
- It is denoted by sigma (σ).
- Notation: $\sigma p(R)$ ○ **Where** , σ is used for selection prediction

R is used for relation **p** is used as a propositional logic formula which may use connectors like: AND OR and NOT. These relational can use as relational operators like =, $\neq$, $\geq$, $<$, $>$, $\leq$.

Example σ PATIENT_NAME="Atharva" (PATIENT)

2. Project Operation:

- This operation shows the list of those attributes that we wish to appear in the result. Rest of the attributes are eliminated from the table.
- It is denoted by Π .
- **Where** **A1, A2, A3** is used as an attribute name of relation **R**.

Example Π PATIENT_NAME, PATIENT_AGE(PATIENT)

3. Union Operation:

- Suppose there are two tuples R and S. The union operation contains all the tuples that are either in R or S or both in R & S.
- It eliminates the duplicate tuples. It is denoted by $\cup$.
- R and S must have the attribute of the same number.
- Duplicate tuples are eliminated automatically.

Example $\sqcup$ PATIENT_NAME (PATIENT_ID) $\cup$ $\sqcup$ DOCTOR_NAME (DOCTOR_ID)

4. Set Intersection:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in both R & S.
- It is denoted by intersection $\cap$.

Example $\sqcap$ PATIENT_NAME (PATIENT_ID) $\cap$ $\sqcap$ DOC_NAME (SUPPORTING_STAFF)

5. Set Difference:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in R but not in S.
- It is denoted by intersection minus ($-$).

Example $\sqcap$ STUDENT_NAME (TSEC_STUDENT) $-$ $\sqcap$ EMP_NAME (IT_STUDENT)

6. Rename Operation:

The rename operation is used to rename the output relation. It is denoted by **rho** (ρ).

Example Rename STUDENT relation to NEWSTUDENT. ρ

NEWSTUDENT(STUDENT)

7. Cross Product/Cartesian product Operations:

CARTESIAN PRODUCT take every tuple one by one from the left set(relation) and will pair it up with all the tuples in the right set(relation).

So, the CROSS PRODUCT of two relation A(R1, R2, R3, ..., Rp) with degree p, and B(S1, S2, S3, ..., Sn) with degree n, is a relation C(R1, R2, R3, ..., Rp, S1, S2, S3, ..., Sn) with degree p + n attributes.

Note: the two relations on which we are performing the operations do not have the same type of tuples, which means Union compatibility (or Type compatibility) of the two relations is not necessary.

Notation:

$A \times S$

the symbol 'X' is used to denote the CROSS PRODUCT operator.

Consider relations

STUDENT(SNO, FNAME, LNAME)

DETAIL(ROLLNO, AGE)

STUDENT

SNO	FNAME	LNAME
1	Inaaya	Malik
2	Bhoomi	Desai

DETAIL

ROLLNO	AGE
10	17
11	18

On applying CROSS PRODUCT on STUDENT and DETAIL

SNO	FNAME	LNAME	ROLLNO	AGE
-----	-------	-------	--------	-----

1	Inaaya	Malik	10	17
1	Inaaya	Malik	11	18
2	Bhoomi	Desai	10	17
2	Bhoomi	Desai	11	18

li.e.number of tuples in STUDENT relation is 2, and the number of tuples in DETAIL is 2. So the number of tuples in the resulting relation on performing CROSS PRODUCT is $2 \times 2 = 4$.

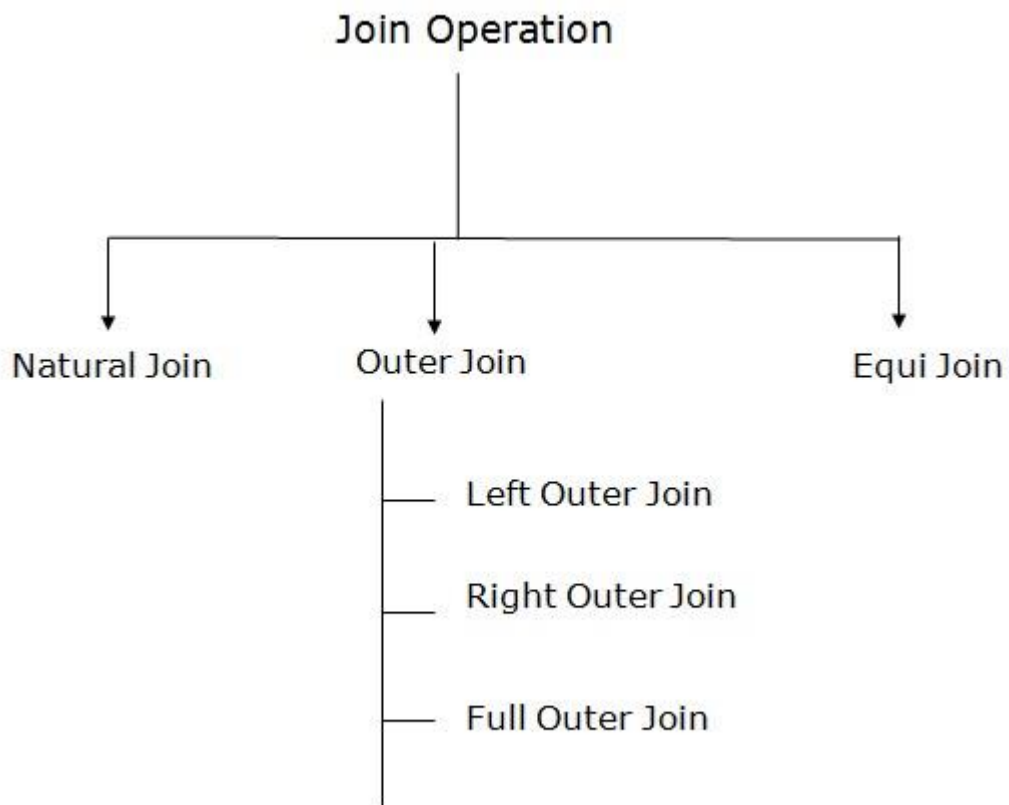
Important points on CARTESIAN PRODUCT(CROSS PRODUCT) Operation:

1. The cardinality (number of tuples) of resulting relation from a Cross Product operation is equal to the number of tuples (say m) in the first relation multiplied by the number of tuples in the second relation (say n).
Cardinality = $m \times n$
2. The Cross Product of two relation A(R1, R2, R3, ..., Rp) with degree p, and B(S1, S2, S3, ..., Sn) with degree n, is a relation C(R1, R2, R3, ..., Rp, S1, S2, S3, ..., Sn) with degree $p + n$ attributes.
Degree = $p + n$

8.Join Operations:

A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied.

It is denoted by $\bowtie$.



Natural Join ($\bowtie$)

Natural join is a binary operator. Natural join between two or more relations will result set of all combination of tuples where they have equal common attribute.

Let us see below example

Emp			Dep	
(Name	Id	Dept_name)	(Dept_name	Manager)
-----			-----	
Smith	120	IT	Sale	Rohit
Jenifer	125	HR	Prod	Devashish
Rayan	110	Sale	IT	Smith
Mita	111	IT		

Emp $\bowtie$ Dep

Name	Id	Dept_name	Manager
------	----	-----------	---------

```

-----
Smith      120    IT      Smith
Rayan      110    Sale     Rohit
Mita       111    IT       Smith

```

Conditional Join

Conditional join works similar to natural join. In natural join, by default condition is equal between common attribute while in conditional join we can specify the any condition such as greater than, less than, not equal

Let us see below example

R			S		
(ID	Sex	Marks)	(ID	Sex	Marks)
-----			-----		
1	F	45	10	M	20
2	F	55	11	M	22
			3	F	60
12	M	59			

Join between R And S with condition **R.marks >= S.marks**

R.ID	R.Sex	R.Marks	S.ID	S.Sex	S.Marks

1	F	45	10	M	20
1	F	45	11	M	22
2	F	55	10	M	20
2	F	55	11	M	22
3	F	60	10	M	20
3	F	60	11	M	22
3	F	60	12	M	59

There are three kinds of outer joins – left outer join, right outer join, and full outer join.

Left Outer Join(R ⋈ S)

All the tuples from the Left relation, R, are included in the resulting relation. If there are tuples in R without any matching tuple in the Right relation S, then the S-attributes of the resulting relation are made NULL.

Left			
A		B	
100		Database	
101		Mechanics	
102		Electronics	
Right			
C		D	
100		Alex	
102		Maya	
104		Mira	
Courses  HoD			
A	B	C	D
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya

Right Outer Join: (R $\bowtie$ S)

All the tuples from the Right relation, S, are included in the resulting relation. If there are tuples in S without any matching tuple in R, then the R-attributes of resulting relation are made NULL.

Courses $\bowtie$ HoD			
A	B	C	D
100	Database	100	Alex
102	Electronics	102	Maya
---	---	104	Mira

Full Outer Join: (R $\bowtie$ S)

All the tuples from both participating relations are included in the resulting relation. If there are no matching tuples for both relations, their respective unmatched attributes are made NULL.

Courses $\bowtie$ HoD			
A	B	C	D
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya
---	---	104	Mira

Anomalies

There are different types of anomalies which can occur in referencing and referenced relation

STUDENT

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNT RY	STUD_AGE
1	RAM	9716271721	Haryana	India	20
2	RAM	9898291281	Punjab	India	19
3	SUJIT	7898291981	Rajsthan	India	18
4	SURESH		Punjab	India	21

Table 1

STUDENT_COURSE

STUD_NO	COURSE_NO	COURSE_NAME
1	C1	DBMS
2	C2	Computer Networks
1	C2	Computer Networks

Table 2

Insertion anomaly: If a tuple is inserted in referencing relation and referencing attribute value is not present in referenced attribute, it will not allow inserting in referencing relation. For Example, If we try to insert a record in STUDENT_COURSE with STUD_NO =7, it will not allow.

Deletion and Updation anomaly: If a tuple is deleted or updated from referenced relation and referenced attribute value is used by referencing attribute in referencing relation, it will not allow deleting the tuple from referenced relation. For Example, If we try to delete a record from STUDENT with STUD_NO =1, it will not allow.

To avoid this, following can be used in query:

- **ON DELETE/UPDATE SET NULL:** If a tuple is deleted or updated from referenced relation and referenced attribute value is used by referencing attribute in referencing relation, it will delete/update the tuple from referenced relation and set the value of referencing attribute to NULL.
- **ON DELETE/UPDATE CASCADE:** If a tuple is deleted or updated from referenced relation and referenced attribute value is used by referencing attribute in referencing relation, it will delete/update the tuple from referenced relation and referencing relation as well.